

Déploiement de l'application

Pour La mise en place du déploiement de mon application j'ai choisi d'utiliser Vercel. Vercel est une plateforme optimisée pour les applications Frontend et les Serverless Function.

Qu'est-ce qu'une Serverless Fonctions ?

Les serverless fonctions de Vercel sont des fonctions backend exécutées à la demande, sans serveur à gérer. Elles représentent une architecture moderne où le code backend est déployé sous forme de fonctions individuelles qui sont invoquées uniquement lorsqu'une requête HTTP est reçue.

Le concept principal de Vercel est de transformer l'application en serverless fonctions au lieu d'avoir un serveur qui tourne en permanence, chaque requête API déclenche une fonction serverless qui s'exécute à la demande.

Architecture de déploiement :

```
vite_gourmand/
├── frontend/ ..... # Application React
│   ├── src/
│   ├── public/
│   └── package.json
├── back/ ..... # Backend Express
│   ├── routes/api/
│   ├── config/
│   └── models/
├── api/ ..... # Serverless functions Vercel
│   ├── [...route].js # Catch-all handler
│   └── index.js ..... # MongoDB connection
└── vercel.json ..... # Configuration Vercel
...
```

Le flux des requêtes :

1. Utilisateur : <https://vite-gourmand.vercel.app/api/menus>
2. Vercel : route la requête vers api/[...route].js
3. Handler : Normalise l'URL et appel Express
4. Express : traite la route api/menus
5. Route : interroge PostgreSQL via le pool de connexion
6. Réponse : Retourne les données au client

Logique de fonctionnement :

Vercel utilise un système de routage qui redirige vers les bonnes fonctions serverless.

Exemple de routage depuis le fichier vercel.json .

```
{
  "rewrites": [
    {
      "source": "/api/:path*",
      "destination": "/api/[...route]"
    }
  ]
}
```

Toute requête commençant par /api est redirigée vers api/[...route].js

Le [...route] est un catch-call qui capture tout le chemin après /api/ exemple: /api/menus → api/[...route].js reçoit req.url = /menus

Le handler serverless (api/[...route]) → Ce fichier est le point d'entrée unique pour toutes les routes API. Il fait le pont entre Express et Vercel.

On commence par normaliser l'url Vercel passe /menus mais Express attend `/api/menus`.

```
module.exports = async (req, res) => {
  // 1. Normaliser l'URL
  // Vercel passe "/menus" mais Express attend "/api/menus"
  let originalUrl = req.url || "/";
  if (!originalUrl.startsWith("/api")) {
    originalUrl = `/api${originalUrl}`;
  }

  // 2. Reconstruire les propriétés Express
  req.url = originalUrl;
  req.path = parsedUrl.pathname;
  req.query = parsedUrl.query;

  // 3. Appeler l'application Express
  return app(req, res);
};
```

On reconstruit les propriétés Express

```
// 2. Reconstruire les propriétés Express
req.url = originalUrl;
req.path = parsedUrl.pathname;
req.query = parsedUrl.query;
```

On appelle l'application Express

```
// 3. Appeler l'application Express
return app(req, res);
```

Cette normalisation permet de reconstruire URL complète avant de passer à Express.

Montage des routes Express :

Dans api/[...route].js toutes les routes sont montées sur l'application Express

```
// Montage de toutes les routes
console.log("\ Montage des routes API...");
app.use("/api/auth", authRouter);
app.use("/api/admin", adminRouter);
app.use("/api/avis", avisRouter);
console.log("✅ Route /api/avis montée");
app.use("/api/commandes", commandesRouter);
app.use("/api/contact", contactRouter);
app.use("/api/dashboard/user", dashboardUserRouter);
app.use("/api/employe", employeRouter);
app.use("/api/horaires", horairesRouter);
app.use("/api/menus", menusRouter);
app.use("/api/plats", platsRouter);
app.use("/api/roles", rolesRouter);
app.use("/api/user", userRouter);
console.log("✅ Toutes les routes montées");
```

Chaque route (ex: authRouter) est définie dans back/routes/api/ Express match l'URL et appelle le bon handler, les routes fonctionnent exactement comme en développement local.

Pré-déploiement :

- Exporter les données depuis MySQL
- Créer les tables dans Supabase à travers le fichier (migration-postgresql-v001.sql)
- Importer les données (export-postgresql-data.sql)
- Synchroniser les séquences (fix-postgresql-sequences.sql)

Configuration Vercel :

- Créer un compte sur Vercel et création du projet
- Importer le GitHub depuis la branche main
- Configurer les variables d'environnements :
 1. `DB_TYPE =postgres`
 2. `DATABASE_URL`
 3. `JWT_SECRET`
 4. `MONGODB_URI`
 5. `VITE_API_URL`
 6. `SUPABASE_URL`
 7. `SUPABASE_SERVICE_ROLE_KEY`
 8. `MONGODB`
 9. Vérification que le fichier vercel.json est bien correct côté routage

7. Vérification que api/[...route].js existe

```
{
  "buildCommand": "cd frontend && npm install && npm run build",
  "outputDirectory": "frontend/dist",
  "rewrites": [
    {
      "source": "/api/paths*",
      "destination": "/api/[...route]"
    },
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ],
  "functions": {
    "api/[...route].js": {
      "maxDuration": 30
    }
  }
}
```

Rappel pré-déploiement :

L'application est en production 100% sur Supabase (PostgreSQL). Les étapes d'export MySQL et d'import (migration-postgresql-v001.sql, export-postgresql-data.sql, fix-postgresql-sequences.sql) ont été réalisées une fois lors de la migration initiale. Pour un déploiement actuel, la base Supabase est déjà en place, il suffit de configurer les variables d'environnement sur Vercel et de déployer.

Accès aux variables d'environnement sur Vercel :

Variable	Description
SUPABASE_SERVICE_ROLE_KEY	Clé « service rôle » (upload images, etc.)
SUPABASE_URL	URL du projet Supabase
FRONTEND_URL	URL du front pour les liens dans les emails
DATABASE_URL	URL de connexion Supabase (PostgreSQL)
VITE_API_URL	URL de l'API appelée par le front (après déploiement)
SMTP_FROM	Adresse affichée comme expéditeur
SMTP_PASSWORD	Mot de passe d'application : Pour Gmail il faut utiliser un mot de passe d'application et non le mot de passe du compte
SMTP_USER	Adresse email utilisé pour l'envoi
SMTP_SECURE	False
SMTP_PORT	Le port SMTP
SMTP_HOST	Serveur SMTP (envoi d'e-mails)
DB_TYPE	postgres
MONGODB_URI	URI de connexion à la base MongoDB
JWT_SECRET	Clé secrète pour signer et vérifier les tokens

Test post-déploiement :

- Test de santé API : api/health
- Test de base de données : api/test-db
- Test d'inscription : api/auth/register
- Test de connexion utilisateur : api/auth/login
- Test de récupération menus : api/menus
- Test de récupération des avis : api/avis/public
- Vérifier les logs Runtime dans Vercel avec le code des statuts

Résumé du flux complet :

```
1. Développement local
  . . ↳ Express.js sur port 3000
  . . ↳ MySQL ou PostgreSQL selon DB_TYPE

2. Build Vercel
  . . ↳ Frontend : npm run build → frontend/dist/
  . . ↳ Backend : api/[...route].js reste tel quel

3. Déploiement
  . . ↳ Vercel déploie frontend/dist/ comme site statique
  . . ↳ Vercel transforme api/[...route].js en serverless function

4. Requête utilisateur
  . . ↳ GET /api/menus
  . . ↳ Vercel route vers api/[...route].js
  . . ↳ Handler normalise URL → /api/menus
  . . ↳ Express matche la route
  . . ↳ Route interroge PostgreSQL
  . . ↳ Réponse JSON retournée
```